

necessary for you to install JDK or Selenium. This is because the API leverages ChromeDriver installation on the system.

First, you should add the laravel/dusk composer dependency to your application as shown in the below listing:

```
composer require --dev laravel/dusk
```

The next step is to register the `Laravel\Dusk\DuskServiceProvider` service provider via the automated service provider registration service provided by the Laravel framework. Once the installation of the Dusk package has been completed, you need to run the `dusk:install` Artisan command:

```
php artisan dusk:install
```

As soon as you execute the above command, a new browser directory will be created with a sample test. The `APP_URL` environment variable should be configured in your `.env` file, which should perfectly match the URL used to access the application from a browser. You can also create a `env.dusk.{environment}` file to force Dusk to use its own environment file. The system will automatically perform a backup and rename your `.env` file. After the completion of the test, the relevant `.env` file will be restored.



Laravel Dusk API simplifies the testing process

You should use the `php artisan dusk:make LoginTest` command to generate a Dusk test. The output will be placed inside the `tests/Browser` directory. You can now execute the tests by using the `php artisan dusk` command, which accepts arguments as accepted by the PHPUnit test runner.

If you work with Laravel Dusk for the first time, the tool makes use of Google Chrome and ChromeDriver to run your browser tests. If you install and start your own Selenium server, you will be able to run all the tests with a browser of your choice. For this purpose, you should navigate to the `tests/DuskTestCase.php` file and remove all references to the `startChromeDriver()` method, which will force the Dusk to start the